

```
COMMAND  PID  USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
python  25737 jarvis  3u  IPv4 901409      0t0  TCP *:http-alt (LISTEN)
```

The above output shows that Python is listening on Port 8080.

## Process management commands

### 1. *nohup*

There are often times when we want to execute long running jobs that are immune to log outs, or wish to terminate an unplanned login session. For such cases, the *nohup* command comes to the rescue. It runs a command and makes it immune to hangups, with the output to a non-tty device. *nohup* does not disconnect a command from the terminal; it makes the command ignore SIGHUP, and redirects stdout/stderr to a file *nohup.out*, so that the command can continue running in the background even when the user logs out.

To understand this, let us create a simple script that prints numbers from 1 to n in a loop:

```
$ cat counter.sh
#!/bin/bash

cnt=1

while [ 1 ];
do
    echo "Current counter value = ${cnt}"
    let ++cnt
    sleep 1
done
```

Now make this script executable and run it in the background with the *nohup* command:

```
$ chmod +x counter.sh
$ nohup ./counter.sh&    # Please note ampersand(&) symbol at
the end of command. This will run script in background
```

After the successful execution of this command, the following output is shown on the terminal:

```
[1] 3745
ignoring input and appending output to 'nohup.out'
```

In the above command, 3745 is the PID of the *counter.sh* script. Now, exit from the terminal, open a new terminal and check whether *counter.sh* script is still running.

```
$ pgrep counter.sh
3745
```

The above output shows that the script is still running with the same PID.

### 2. *pidof*

The *pidof* command finds the process IDs (pids) of the named programs. It prints these IDs on the standard output. For instance, if we want to find the process ID of bash, then we can execute the command as follows:

```
$ pidof bash
28711 20899 20891 5960
```

I have four instances of bash, and hence there are four process IDs listed.

To list the process ID of a script, we have to use the *-x* option. Let us first create a script as follows:

```
$ cat pidof-demo.sh
#!/bin/bash

while [ 1 ];
do
    echo "script for demo of pidof command"
    sleep 1
done
```

Now, make it executable and run it in a terminal as follows:

```
$ chmod +x pidof-demo.sh
$ ./pidof-demo.sh
```

Now execute the *pidof* command from another terminal with the *-x* option, as follows:

```
$ pidof -x pidof-demo.sh
22211
```

The above output shows 22211 as a process ID of the *pidof-demo.sh* shell script.

## Docker commands

Container technologies are getting more popular these days. We often see containers in production as well. Docker is one of the most mature and popular container technologies in the market. Hence it is worthwhile to discuss a few Docker related commands in this article. Let us begin with a few useful ones.

### 1. *docker exec*

*docker exec* allows us to execute commands inside a running container. Given below is the syntax of the same:

```
docker exec [OPTIONS] CONTAINER COMMAND
```